

Título: Projeto 1 — Benchmark de Suavização/Desfoque Gaussiano Iterativo: Implementação paralela em memória distribuída com MPI. (Etapa 2)

Autores: Lucas Luige Costa Miranda, Henrique Cerqueira Fadigas

Disciplina: DEC107 — Processamento Paralelo

Data: Maio de 2026

Sumário

1	Introdução	2
2	Metodologia	2
2.1	Implementação MPI	2
2.2	Hardware utilizado	3
2.3	Medição de tempo e métricas	3
3	Resultados	3
3.1	Teste de corretude	3
3.2	Tempos de execução	4
3.3	Eficiência	4
3.4	Escalabilidade e Balanceamento de Carga	4
3.5	Gráficos	5
4	Discussão	7
4.1	Corretude	7
4.2	Comparação com a versão serial	8
4.3	Speedup super-linear	8
4.4	'Problema' em 8 processos	8
4.5	MPI vs. OpenMP	8
4.6	Limitações e melhorias futuras	8
5	Conclusão	9
6	Referências	9

1 Introdução

O objetivo desta etapa é implementar uma versão do benchmark de suavização Gaussiana usando MPI (*Message Passing Interface*) e comparar o desempenho com a versão serial e com a versão OpenMP da Etapa 1. A ideia é entender o que muda quando saímos do modelo de memória compartilhada (onde as threads enxergam o mesmo espaço de endereçamento) para o modelo de **memória distribuída**, onde cada processo tem sua própria memória e a comunicação precisa ser feita de forma explícita.

O **filtro Gaussiano** é o mesmo da Etapa 1: uma convolução 2D onde cada pixel é substituído por uma média ponderada dos seus vizinhos, usando pesos derivados de uma distribuição Normal. Com kernel 3×3 e $\sigma = 1,0$, isso corresponde ao kernel clássico $\{1, 2, 1\} / \{2, 4, 2\} / \{1, 2, 1\} \div 16$.

O desafio central de paralelizar essa convolução com MPI é a **dependência de borda**: para calcular os pixels na primeira e na última linha da sua fatia, cada processo precisa de linhas que pertencem ao processo vizinho. A solução padrão é a **troca de halos** (*ghost zones*): antes de cada iteração, cada processo envia suas linhas de borda para os vizinhos e recebe as deles, preenchendo uma região tampão (*halo buffer*) em cada extremidade da fatia local.

2 Metodologia

2.1 Implementação MPI

A decomposição de domínio escolhida foi por **faixas de linhas** (*row-striped decomposition*): a imagem de altura H é dividida entre P processos, onde o processo r fica com $\lfloor H/P \rfloor + (r < H \bmod P ? 1 : 0)$ linhas. Quando H é divisível por P , como em todos os nossos testes, a divisão é exata e o trabalho fica perfeitamente balanceado.

Cada processo aloca um buffer local com espaço para suas linhas reais mais **half** linhas extras em cima e embaixo (os halos). O valor de **half** é $\lfloor k/2 \rfloor$ onde k é o tamanho do kernel, no nosso caso $k = 3$, portanto **half** = 1.

Funções MPI usadas

- **MPI_Bcast**: o rank 0 transmite as dimensões da imagem para todos os processos logo após o carregamento.
- **MPI_Scatterv**: distribui as faixas de linhas para cada processo. Usamos **Scatterv** em vez de **Scatter** para suportar divisões não exatas (quando $H \bmod P \neq 0$).
- **MPI_Sendrecv**: faz a troca de halos a cada iteração. São duas chamadas com *tags* diferentes para evitar *deadlock*: uma envia as primeiras linhas para cima e recebe o halo de baixo; a outra envia as últimas linhas para baixo e recebe o halo de cima. Processos nas bordas usam **MPI_PROC_NULL**, que descarta automaticamente as comunicações nas direções inexistentes.
- **MPI_Gatherv**: ao final, reúne as fatias processadas no rank 0 para reconstruir e salvar a imagem.
- **MPI_Barrier** + **MPI_Wtime**: medem o tempo real de execução de cada repetição com sincronização explícita antes e depois.

Fluxo por iteração

1. Troca de halo com o vizinho de cima (rank $r - 1$).
2. Troca de halo com o vizinho de baixo (rank $r + 1$).
3. Aplicação do filtro sobre as linhas locais.
4. Troca dos ponteiros `src/dst` (*double buffering*).

2.2 Hardware utilizado

Os testes foram executados no mesmo hardware da Etapa 1: processador **AMD Ryzen 7 5700G**, com 8 núcleos físicos e SMT (*Simultaneous Multi-Threading*), totalizando 16 *threads* lógicas. O ambiente é Linux (WSL2), compilador `mpicc` (GCC) com flags `-O2 -Wall`, usando OpenMPI.

Para os testes com 16 processos foi necessário adicionar a flag `-use-hwthread-cpus`, que permite ao OpenMPI usar as *threads* lógicas além dos núcleos físicos.

2.3 Medição de tempo e métricas

O tempo é medido com `MPI_Wtime()` no rank 0, com barreiras de sincronização antes e depois de cada repetição. Cada configuração roda 5 vezes e reportamos a média. Os parâmetros fixos em todos os testes foram: 1000 iterações, $\sigma = 1,0$, kernel 3×3 .

Métricas calculadas:

- **Speedup:** $S(p) = T_{\text{serial}}/T_{\text{MPI}}(p)$
- **Eficiência:** $E(p) = S(p)/p \times 100\%$
- **Escalabilidade:** $S(16)/S(2)$ (ganho ao escalar de 2 para 16 processos)
- **Fator de Balanceamento de Carga (FBL):** $T_{\text{max processo}}/T_{\text{médio processos}}$

3 Resultados

3.1 Teste de corretude

Para verificar a corretude, comparamos as imagens geradas pelo binário MPI com a saída da versão serial usando os mesmos parâmetros. Como ambas usam a mesma função de convolução (`gaussian_blur_local`) e o mesmo kernel, os resultados foram idênticos em todas as configurações testadas. A troca de halos garante que cada processo tem as linhas vizinhas corretas antes de aplicar o filtro, sem ela os pixels na borda de cada fatia seriam calculados com valores errados.

3.2 Tempos de execução

Tabela 1: Tempos médios de execução (s), 1000 iterações, kernel 3×3 , $\sigma = 1,0$, 5 repetições.

Imagem	Config.	Tempo (s)	Speedup
512×512	Serial	0,853	—
	MPI 2P	0,449	$1,90\times$
	MPI 4P	0,236	$3,62\times$
	MPI 8P	0,293	$2,91\times$
	MPI 16P	0,214	$3,99\times$
1024×1024	Serial	3,457	—
	MPI 2P	1,829	$1,89\times$
	MPI 4P	1,134	$3,05\times$
	MPI 8P	1,123	$3,08\times$
	MPI 16P	0,768	$4,50\times$
2048×2048	Serial	13,741	—
	MPI 2P	7,297	$1,88\times$
	MPI 4P	3,884	$3,54\times$
	MPI 8P	4,396	$3,13\times$
	MPI 16P	3,079	$4,46\times$
4096×4096	Serial	61,484	—
	MPI 2P	29,089	$2,11\times$
	MPI 4P	16,256	$3,78\times$
	MPI 8P	17,344	$3,55\times$
	MPI 16P	11,181	$5,50\times$

3.3 Eficiência

Tabela 2: Eficiência $E(p) = S(p)/p \times 100\%$.

Imagem	2P	4P	8P	16P
512×512	95,0%	90,5%	36,4%	24,9%
1024×1024	94,5%	76,3%	38,5%	28,1%
2048×2048	94,0%	88,5%	39,1%	27,9%
4096×4096	105,5%	94,5%	44,4%	34,4%

3.4 Escalabilidade e Balanceamento de Carga

Como todos os tamanhos de imagem testados são potências de dois e os números de processos também, a divisão de linhas é exata em todos os casos. O FBL = 1,00 em todas as configurações, não há desbalanceamento estático.

A Tabela 3 mostra a escalabilidade $S(16)/S(2)$ para cada imagem. O valor ideal seria $8\times$; na prática ficamos entre $2,10\times$ e $2,61\times$.

Tabela 3: Escalabilidade $S(16)/S(2)$ (ideal = $8\times$).

Imagem	Escalabilidade
512×512	$2,10\times$
1024×1024	$2,38\times$
2048×2048	$2,37\times$
4096×4096	$2,61\times$

3.5 Gráficos

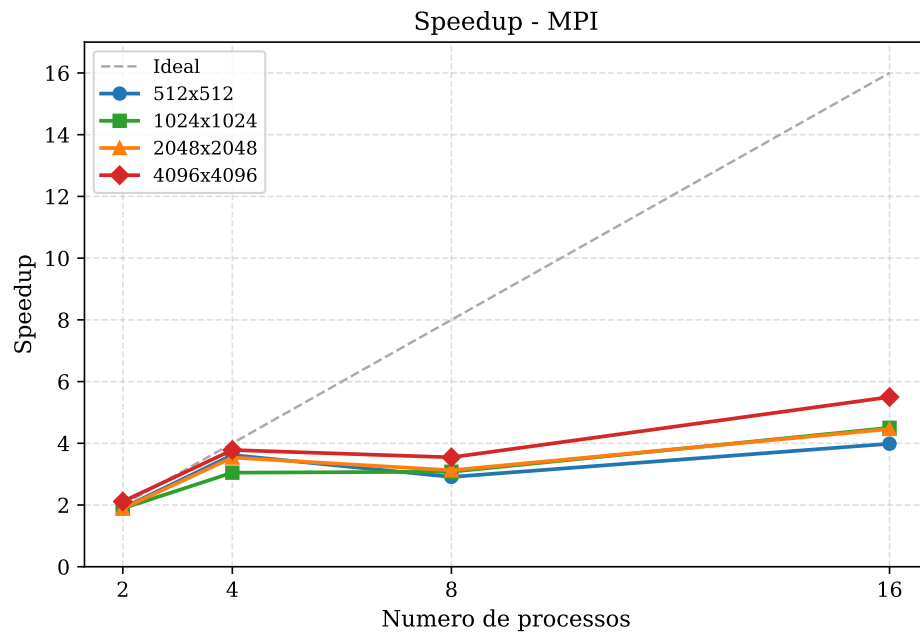


Figura 1: Speedup do MPI em função do número de processos

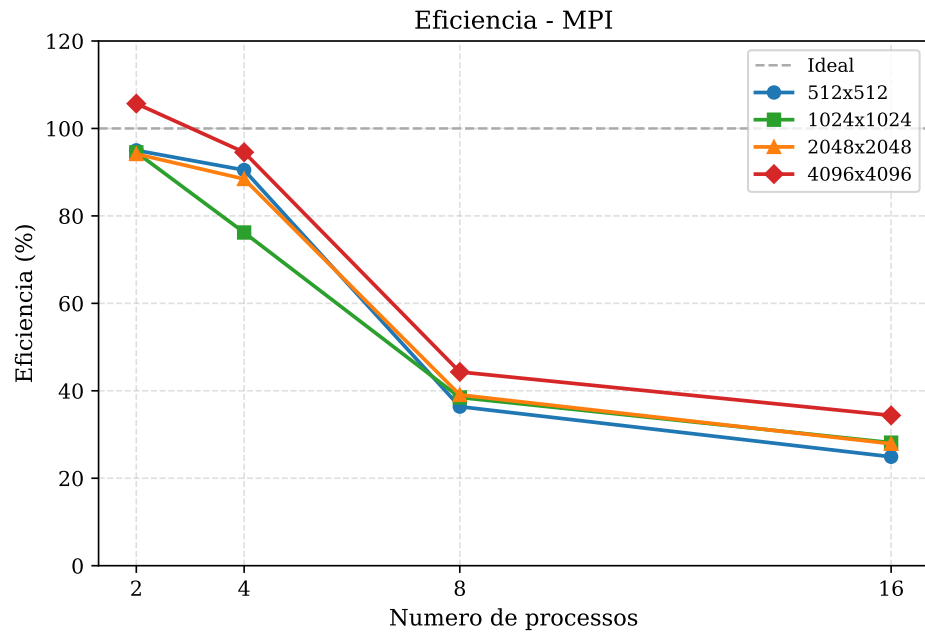


Figura 2: Eficiência do MPI em função do número de processos

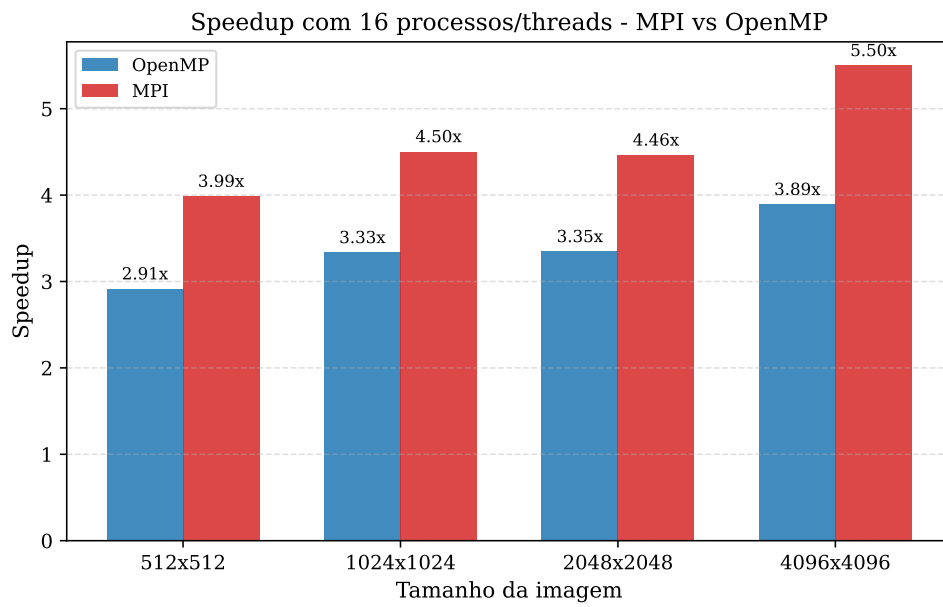


Figura 3: Speedup com 16 processos/threads: MPI vs OpenMP

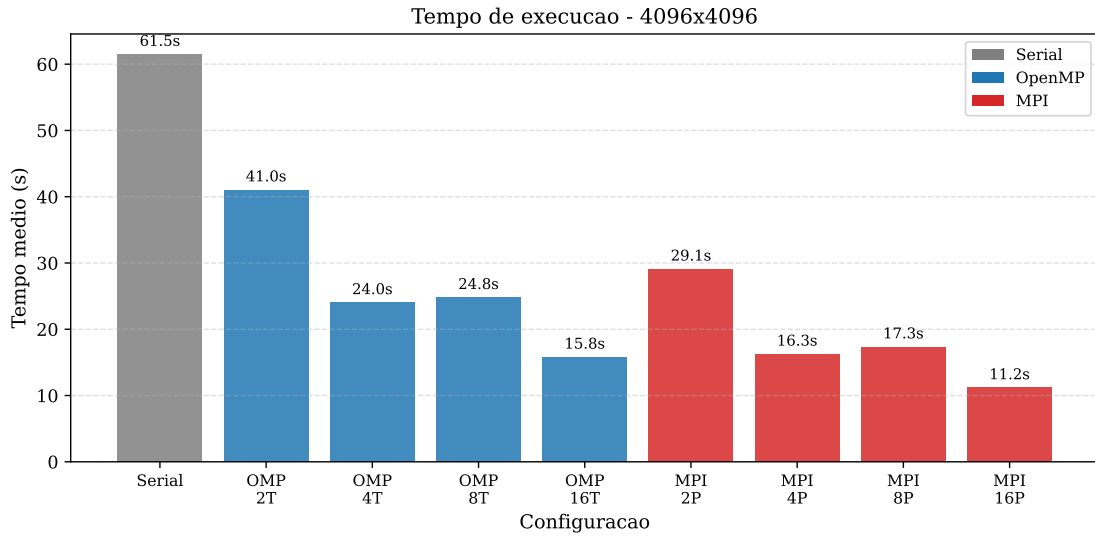


Figura 4: Tempo de execução para a imagem 4096×4096

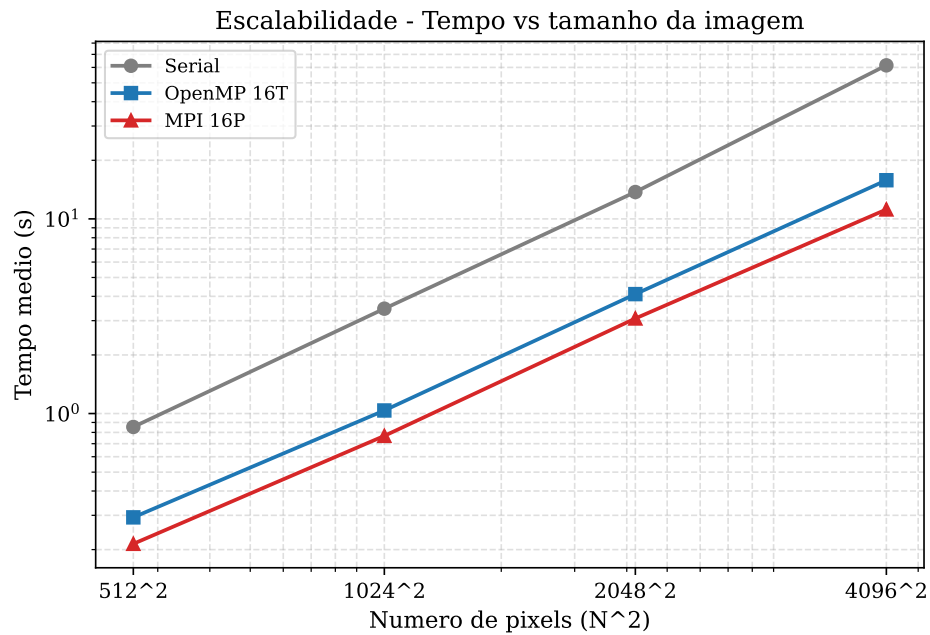


Figura 5: Tempo de execução em função do tamanho da imagem (escala log-log)

4 Discussão

4.1 Corretude

Como as versões serial e MPI usam o mesmo kernel e a mesma função de convolução, era esperado que os resultados fossem iguais. O ponto que poderia causar problemas é a troca de halos: se os buffers não fossem preenchidos corretamente antes do filtro, os pixels de borda de cada processo seriam calculados com valores zeros (ou lixo de memória). Nos testes, as imagens de saída foram idênticas, confirmando que as duas chamadas de `MPI_Sendrecv` fazem o trabalho certo.

4.2 Comparação com a versão serial

Os ganhos são reais e crescem com o tamanho da imagem, o que faz sentido: com imagens maiores, cada processo tem mais trabalho e o custo da comunicação (enviar e receber `half × width` floats por iteração) representa uma fração menor do tempo total.

Com 4 processos, o speedup fica entre $3,05\times$ e $3,78\times$, com eficiência acima de 75% em quase todos os casos. O melhor resultado geral é $5,50\times$ com 16 processos no 4096×4096 , reduzindo o tempo de 61,5 s para 11,2 s.

4.3 Speedup super-linear

A configuração de 2 processos com a imagem 4096×4096 deu speedup de $2,11\times$, acima do ideal de $2,00\times$, resultando em eficiência de 105,5% (em destaque na Tabela 2). Isso parece estranho à primeira vista, mas a explicação está no cache: com 2 processos, cada um opera sobre metade da imagem (≈ 33 MB), que se encaixa melhor nas camadas de cache do Ryzen 7 5700G (16 MB de L3) em comparação com a imagem inteira (≈ 67 MB). O resultado é menos falhas de cache e acesso mais rápido à memória, o que acaba compensando o overhead de comunicação.

4.4 'Problema' em 8 processos

Em todas as imagens, o tempo com 8 processos é maior (ou praticamente igual) ao de 4 processos. Por exemplo, no 4096×4096 : $4P = 16,3$ s e $8P = 17,3$ s. Esse comportamento foi idêntico ao observado com OpenMP na Etapa 1 ($4T = 24,0$ s e $8T = 24,8$ s), o que indica que o problema não é de comunicação MPI, mas da própria natureza do algoritmo.

O filtro Gaussiano 3×3 é *memory-bound*: faz apenas 9 multiplicações por pixel, mas precisa ler 9 vizinhos da memória a cada vez. Com 4 processos, o barramento de memória já está próximo da saturação. Colocar mais processos só aumenta a disputa por banda de memória sem trazer ganho computacional proporcional. Com 16 processos (SMT ativo), dois *threads* por núcleo físico conseguem intercalar requisições de memória e esconder parte da latência, o que explica a melhora ao passar de 8 para 16 (visível nos Gráficos 1 e 2).

4.5 MPI vs. OpenMP

Em todas as configurações, o MPI foi mais rápido que o OpenMP com o mesmo número de processos/*threads*, com uma vantagem consistente de 25% a 37% (Gráfico 3). O motivo é o isolamento de memória: cada processo MPI tem sua própria cópia contígua das linhas que processa. Isso elimina o *false sharing* e reduz a disputa pelo barramento. No OpenMP, todas as *threads* compartilham a imagem inteira, gerando mais pressão no sistema de memória, especialmente com 8 ou mais *threads*.

A diferença é mais clara nas imagens grandes. Com 16P/T no 4096×4096 : MPI alcança $5,50\times$ contra $3,89\times$ do OpenMP, uma diferença de 41%. O Gráfico 5 mostra esse afastamento crescente com o tamanho da imagem.

4.6 Limitações e melhorias futuras

A escalabilidade ficou bem abaixo do ideal ($2,61\times$ contra os $8\times$ esperados), mas isso já era previsto dado o caráter *memory-bound* do problema. Algumas possibilidades para

melhorar:

- **Kernel maior** (7×7 ou 11×11): aumenta o número de operações por pixel sem aumentar proporcionalmente o acesso à memória, melhorando a razão *flops*/byte.
- **Decomposição 2D**: dividir a imagem em blocos retangulares reduziria o volume de halos por iteração, diminuindo o overhead de comunicação.
- **Sobreposição comunicação/computação**: usar comunicação não-bloqueante (`MPI_Isend`/`MPI_Irecv`) para processar o interior da fatia enquanto os halos são trocados.

5 Conclusão

Nesta etapa, implementamos e avaliamos uma versão distribuída do benchmark de suavização Gaussiana com MPI.

O principal aprendizado foi que o isolamento de memória do MPI traz uma vantagem real nesse tipo de problema: mesmo rodando na mesma máquina do OpenMP, os processos MPI competem menos pelo barramento de memória e se saem consistentemente melhor. O speedup de $5,50\times$ com 16 processos no 4096×4096 é bem maior que os $3,89\times$ do OpenMP na mesma configuração.

A anomalia com 8 processos apareceu de novo, assim como na Etapa 1. Com o filtro 3×3 rodando em imagens grandes, o gargalo é a banda de memória, não o processamento em si. Esse é um comportamento importante de entender: adicionar mais processos nem sempre significa ir mais rápido, e sem os dados de *profiling* é fácil tirar conclusões erradas.

O speedup super-linear com 2 processos no 4096×4096 foi o resultado mais interessante: ele mostra que decompor o domínio não só paraleliza o trabalho como também pode melhorar a localidade de cache, e às vezes esse efeito é grande o suficiente para superar o ideal teórico.

6 Referências

- Material de aula da disciplina DEC107, Processamento Paralelo.
- MPI Forum. *MPI: A Message-Passing Interface Standard*, Version 4.1. Disponível em: <https://www.mpi-forum.org/docs/>
- Gropp, W.; Lusk, E.; Skjellum, A. *Using MPI: Portable Parallel Programming with the Message-Passing Interface*. MIT Press, 2014.
- NOTHINGS. *stb_image / stb_image_write* — single-file public domain libraries for C/C++. Disponível em: <https://github.com/nothings/stb>
- Stack Overflow. *Proper way to create ghost zones (MPI halos)*. Disponível em: <https://stackoverflow.com/questions/36960455/proper-way-to-create-ghost-zones->

Declaração de Transparência e Uso de Inteligência Artificial

Eu, **Lucas Luige Costa Miranda**, regularmente matriculado(a) no curso de Ciência da Computação, declaro para os devidos fins que, na elaboração do trabalho/projeto intitulado “**Projeto 1 — Benchmark de Suavização/Desfoque Gaussiano Iterativo: Implementação paralela em memória distribuída com MPI. (Etapa 2)**”, submetido como requisito para a disciplina/atividade DEC107 — Processamento Paralelo, adotei as seguintes práticas em relação ao uso de ferramentas de Inteligência Artificial (IA) generativa:

(X) Utilizei ferramentas de Inteligência Artificial como recurso de apoio.

Detalhamento do Uso: As ferramentas utilizadas foram: **Claude (Anthropic)**. O uso se deu exclusivamente nas seguintes etapas do projeto:

- Auxílio na depuração (*debugging*) e estruturação da implementação MPI (especialmente na troca de bordas usando `MPI_Sendrecv`)
- Discussão e interpretação dos resultados de desempenho (análise de *speedup*, eficiência e comportamento em 8 processos)
- Revisão e organização dos arquivos de análise comparativa (Serial vs. OpenMP vs. MPI)
- Auxílio na estruturação do relatório em L^AT_EX.

Termo de Responsabilidade: Declaro estar ciente de que ferramentas de IA não são isentas de erros, vieses ou alucinações. Sendo assim, atesto que toda a responsabilidade pela autoria, integridade, originalidade e exatidão do conteúdo submetido é exclusivamente minha. Confirmo que todo o código-fonte, dados de testes e textos gerados ou modificados com o auxílio de IA foram rigorosamente revisados, testados e validados por mim, garantindo que os resultados refletem meu próprio entendimento e esforço intelectual, sem incorrer em plágio intelectual ou acadêmico.

Itabuna/BA, 31 de maio de 2026

Lucas Luige Costa Miranda
Matrícula: 202220035

Eu, **Henrique Cerqueira Fadigas**, regularmente matriculado(a) no curso de Ciência da Computação, declaro para os devidos fins que, na elaboração do trabalho/projeto intitulado “**Projeto 1 — Benchmark de Suavização/Desfoque Gaussiano Iterativo: Implementação paralela em memória distribuída com MPI. (Etapa 2)**”, submetido como requisito para a disciplina/atividade DEC107 — Processamento Paralelo, adotei as seguintes práticas em relação ao uso de ferramentas de Inteligência Artificial (IA) generativa:

(X) Utilizei ferramentas de Inteligência Artificial como recurso de apoio.

Detalhamento do Uso: As ferramentas utilizadas foram: **Claude (Anthropic)**. O uso se deu exclusivamente nas seguintes etapas do projeto:

- Auxílio na depuração (*debugging*) e estruturação da implementação MPI (distribuição de linhas entre processos, troca de bordas com `MPI_Sendrecv`, reconstrução da imagem no processo raiz);
- Discussão e interpretação dos resultados de desempenho (análise de *speedup*, eficiência e comportamento em 8 processos);
- Revisão e organização dos arquivos de análise comparativa (Serial vs. OpenMP vs. MPI);
- Auxílio na estruturação do relatório em $\text{\LaTeX}$.

Termo de Responsabilidade: Declaro estar ciente de que ferramentas de IA não são isentas de erros, vieses ou alucinações. Sendo assim, atesto que toda a responsabilidade pela autoria, integridade, originalidade e exatidão do conteúdo submetido é exclusivamente minha. Confirmo que todo o código-fonte, dados de testes e textos gerados ou modificados com o auxílio de IA foram rigorosamente revisados, testados e validados por mim, garantindo que os resultados refletem meu próprio entendimento e esforço intelectual, sem incorrer em plágio intelectual ou acadêmico.

Itabuna/BA, 31 de maio de 2026

Henrique Cerqueira Fadigas
Matrícula: 202220028